



S16.1 · Píldora: Demostrador y Documentación

Guía práctica para preparar, estructurar y entregar un demostrador funcional y documentación técnica de calidad para proyectos de ML/AI. Desde la elección del framework hasta el checklist de congelación del **10 de diciembre a las 14:30**.

SPRINT 16

ML/AI · DOCUMENTACIÓN

Índice de contenidos

Esta píldora cubre todos los aspectos necesarios para llevar tu proyecto de ML/AI desde el código al demostrador funcional y la documentación entregable.

01

Opciones de demostrador

Streamlit, Gradio o FastAPI

03

README y requisitos

Documentación mínima viable

05

Errores comunes

Qué evitar antes de entregar

02

Estructura del repositorio

Organización de ficheros y carpetas

04

Memoria técnica

Índice sugerido y contenidos

06

Checklist de congelación

10/12 · 14:30 h



BLOQUE 1

Opciones de Demostrador

Elige el framework que mejor se ajuste a tu equipo, a la complejidad del modelo y al tiempo disponible. Cada opción tiene un perfil de uso distinto.

Streamlit, Gradio o FastAPI: ¿cuál elegir?

Streamlit

Ideal para: exploración de datos y dashboards interactivos.

- Curva de aprendizaje muy baja
- Integración nativa con Pandas, Plotly y Matplotlib
- Despliegue rápido en Streamlit Cloud
- Mejor opción si el equipo ya usa Python

Gradio

Ideal para: demos de modelos con entrada/salida multimedia.

- Soporte nativo para imagen, audio, texto y vídeo
- Compatible con Hugging Face Spaces
- Interfaz lista en pocas líneas de código
- Comparte demos con enlace público automático

FastAPI

Ideal para: endpoints de inferencia listos para producción.

- Alto rendimiento con soporte asíncrono
- Documentación Swagger generada automáticamente
- Mejor opción si se integra con frontend existente
- Requiere más configuración inicial

¿Cuándo usar cada opción?



En la mayoría de proyectos de ML académicos o de demostración, **Streamlit** o **Gradio** son la opción más rápida y efectiva. Reserva **FastAPI** cuando necesites un endpoint real de inferencia o cuando el equipo de front ya tenga su propio cliente.

Estructura del Repositorio

Una estructura clara y consistente facilita la revisión del código, la reproducibilidad y la colaboración. Adopta esta organización como estándar mínimo para todos los proyectos.

Estructura recomendada

```
mi-proyecto/
├── app/
│   ├── main.py      # Entry point
│   └── utils.py     # Helpers
├── models/
│   └── model.pkl     # Modelo serializado
├── notebooks/
│   └── exploration.ipynb
├── data/
│   ├── raw/
│   └── processed/
├── tests/
│   └── test_model.py
├── requirements.txt
├── .env.example
└── README.md
```

Principios clave

- Separación de responsabilidades
El código de la app, los modelos y los datos nunca deben mezclarse en la misma carpeta.
- Sin credenciales en el repo
Usa `.env.example` y añade `.env` al `.gitignore` siempre.
- Modelos versionados
Incluye un nombre con versión o fecha en el fichero del modelo:
`model_v1_20241210.pkl`.

README y Requisitos

El README es la primera impresión de tu proyecto. Un README bien escrito permite que cualquier evaluador o colaborador ponga en marcha el demostrador en menos de cinco minutos, sin necesidad de preguntar nada al equipo.

1

Descripción del proyecto

Qué hace el modelo, qué problema resuelve y cuál es el alcance del demostrador. Incluye una captura de pantalla o GIF de la interfaz.

2

Instalación paso a paso

Comandos exactos: `git clone`, creación del entorno virtual, `pip install -r requirements.txt` y variables de entorno necesarias.

3

Cómo ejecutar

Comandos para lanzar el demostrador (`streamlit run app/main.py`) y para llamar al endpoint si aplica.

4

Estructura del repositorio

Árbol de carpetas comentado para que el evaluador entienda dónde está cada componente sin explorar el repo completo.

5

Autoría y licencia

Nombres de los miembros del equipo, fecha y licencia del proyecto (MIT recomendada para proyectos académicos).

El fichero requirements.txt

Congela siempre las versiones exactas de las dependencias para garantizar la reproducibilidad. Usa `pip freeze > requirements.txt` desde el entorno virtual del proyecto, nunca desde el entorno base.

```
# requirements.txt
streamlit==1.32.0
pandas==2.2.1
scikit-learn==1.4.1
numpy==1.26.4
python-dotenv==1.0.1
fastapi==0.110.0    # si usas FastAPI
gradio==4.25.0     # si usas Gradio
```

Separa las dependencias de desarrollo (pytest, black, flake8) en un fichero `requirements-dev.txt` independiente para no contaminar el entorno de producción.

✓ Hacer

- Versiones exactas con `==`
- Entorno virtual dedicado
- Probar en entorno limpio

✗ Evitar

- Dependencias sin versión
- Paquetes instalados globalmente
- Librerías no utilizadas

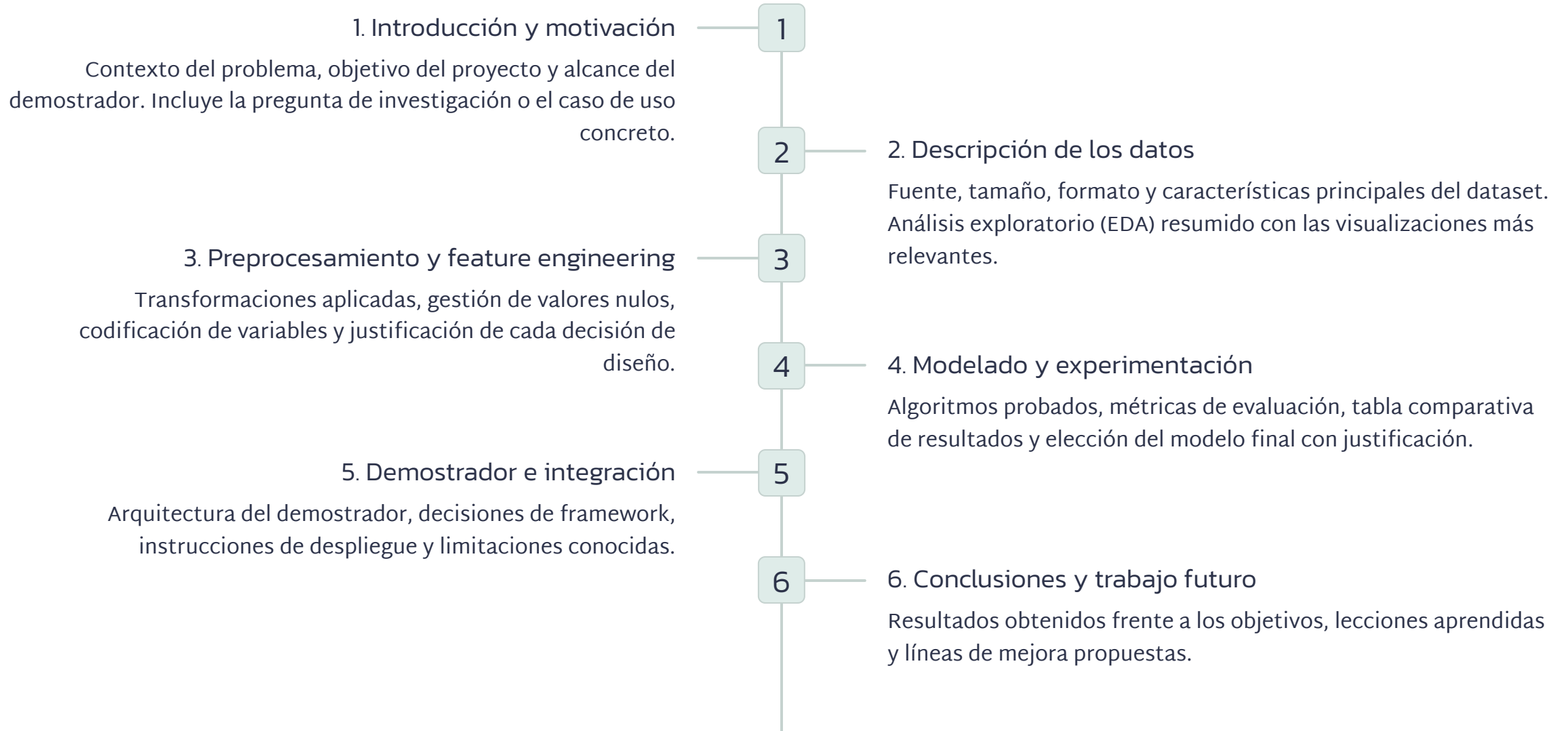


BLOQUE 4

Memoria Técnica: Índice Sugerido

La memoria es el documento que acompaña al código y justifica las decisiones de diseño, datos y modelo. Sigue este índice como punto de partida y adapta las secciones a tu proyecto.

Estructura de la memoria técnica



Errores Comunes: Qué Evitar

Estos son los fallos que aparecen con más frecuencia en los proyectos revisados. Identificarlos a tiempo puede marcar la diferencia entre una entrega sólida y una que no arranca en el equipo evaluador.

● El demostrador no arranca en otro equipo

Causa: dependencias sin versionar o rutas absolutas hardcodeadas. **Solución:** prueba siempre en un entorno limpio antes de la entrega y usa rutas relativas con `pathlib`.

● Credenciales o datos sensibles en el repo

Causa: subir `.env` o ficheros de datos con información personal. **Solución:** revisa el `.gitignore` y usa `git-secrets` como medida de seguridad adicional.

● Modelo no incluido o con ruta incorrecta

Causa: el fichero `.pkl` no está commiteado o la ruta en el código no coincide con la estructura del repo. **Solución:** verifica la ruta de carga en el script principal.

● README incompleto o ausente

Causa: prisa en la entrega. Un README sin instrucciones de instalación obliga al evaluador a reverse-engineerear el proyecto. **Solución:** el README es obligatorio, no opcional.

● Memoria sin justificación de decisiones

Causa: describir qué se hizo sin explicar por qué. **Solución:** por cada elección de modelo, métrica o transformación, incluye al menos una frase de justificación.

BLOQUE 6

Checklist de Congelación

10 dic · 14:30 h

A partir de esta hora no se aceptarán cambios en el repositorio. Repasa cada punto con tu equipo antes de la fecha límite. Un elemento sin marcar es motivo de penalización.

Lista de verificación final

Repositorio y código

- ☐ Estructura de carpetas correcta y limpia
- ☐ Sin rutas absolutas ni ficheros temporales
- ☐ Credenciales fuera del repo (.gitignore)
- ☐ requirements.txt con versiones fijadas
- ☐ Modelo serializado incluido y con ruta correcta
- ☐ Tests básicos ejecutan sin errores

Demostrador

- ☐ App arranca con un solo comando
- ☐ Probado en entorno limpio (no el de desarrollo)
- ☐ Endpoint o interfaz responde correctamente

Documentación

- ☐ README con descripción, instalación y uso
- ☐ Captura o GIF de la interfaz en el README
- ☐ Autoría y licencia incluidas
- ☐ Memoria técnica con todos los apartados del índice
- ☐ Métricas de evaluación documentadas en la memoria
- ☐ Limitaciones y trabajo futuro descritos

Entrega

- ☐ URL del repositorio compartida con el evaluador
- ☐ Última versión commiteada antes de las 14:30 h
- ☐ Tag v1.0-final creado en el repositorio

⚠ Recuerda: el repositorio se congela el **10 de diciembre a las 14:30 h**. Cualquier commit posterior a esa hora no será evaluado.